

ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

COMPUTACIÓN GRÁFICA

INFORME TAREA 02: GeoLab 2D

NOMBRE: GUDIÑO BRYAN

NRC: 36868

INFORME TÉCNICO: SISTEMA INTERACTIVO DE COMPUTACIÓN GRÁFICA Y GEOMETRÍA

1. Introducción

El presente proyecto aborda la problemática que enfrentan las empresas de diseño gráfico al realizar manualmente cálculos de posiciones, áreas, distancias, orientaciones y transformaciones geométricas. Esta práctica manual incrementa la tasa de errores y dificulta la reutilización de diseños.

Para resolver esta necesidad, se diseñó e implementó un sistema informático en C# utilizando el entorno de desarrollo .NET Framework y la biblioteca gráfica GDI+ (System.Drawing). La aplicación permite construir figuras geométricas bidimensionales de manera totalmente algorítmica y paramétrica (sin coordenadas fijas), ofreciendo un entorno interactivo para aplicar transformaciones afines en tiempo real, analizar propiedades vectoriales y comprobar los resultados mediante pruebas integradas bajo un esquema estricto de Programación Orientada a Objetos (POO).

2. Objetivos:

2.1.Objetivo general:

Desarrollar una aplicación de escritorio interactiva en C# con Windows Forms que permita la construcción algorítmica, visualización, transformación matricial y análisis geométrico-vectorial de figuras bidimensionales complejas, aplicando los principios de la computación gráfica 2D.

2.2.Objetivos específicos:

Construcción Algorítmica: Implementar los algoritmos matemáticos y trigonométricos para generar dinámicamente las tres figuras requeridas (Hexágono de 6 colores, Flor de Margarita con pentágonos y Polígono Estrellado de N puntas) sin depender de coordenadas predefinidas.

Análisis Vectorial y Geométrico: Desarrollar un motor matemático desacoplado (`MathEngine`) para calcular magnitudes, direcciones, productos punto, determinantes, áreas (fórmula de Gauss/Shoelace), perímetros y proyecciones ortogonales de puntos sobre segmentos.

Transformaciones Matriciales: Aplicar operaciones de traslación, rotación y escalamiento mediante el uso de matrices en coordenadas homogéneas (3 X 3) sobre un punto pivote central.

Interactividad y Validación: Diseñar una interfaz de usuario fluida con la biblioteca GDI+ que responda en tiempo real a controles paramétricos y eventos del ratón, e integrar un módulo de pruebas automatizadas para validar la precisión de las fórmulas aplicadas.

3. Arquitectura del sistema

El proyecto está estructurado bajo un patrón de diseño que separa la interfaz gráfica de la lógica matemática, garantizando la escalabilidad del código. Los módulos principales son:

- **UI (`MainForm`)**: Gestiona la ventana principal, el ciclo de renderizado continuo (evento `Paint`) y la interacción del usuario mediante teclado y ratón. Implementa "Doble Búfer" mediante reflexión para evitar el parpadeo (*flickering*) durante los redibujados.
- **Models (`FiguraGeometrica, FlorMargarita, HexagonoSeisColores, PoligonoEstrellado`)**: Clases base polimórficas que definen la estructura de cualquier forma, almacenando su centroide, ángulo de rotación y factor de escala.
- **MathEngine (`GeometryMath, Matrix3x3, Vector2D`)**: Motor estático aislado que procesa todas las operaciones algebraicas espaciales (matrices de rotación, proyecciones ortogonales).
- **Testing (`ValidadorMatematico`)**: Módulo de pruebas unitarias integradas que valida la exactitud de los cálculos geométricos y los expone en la interfaz.

4. Fundamentos Matemáticos y Fórmulas

El motor gráfico del sistema no depende de funciones predeterminadas de rotación de UI, sino que calcula las coordenadas exactas de cada vértice en tiempo real.

4.1. Transformaciones Geométricas 2D

Para cada vértice original (x, y) de una figura centrada en el origen $(0,0)$, se aplican tres transformaciones consecutivas para obtener el vértice final en pantalla (x', y'):

4.1.1. **Escalamiento:** Se multiplica por el factor escalar E.

$$x_E = x \cdot E$$
$$y_E = y \cdot E$$

4.1.2. **Rotación:** Se aplica la matriz de rotación por el ángulo θ (convertido a radianes).

$$x_R = x_E \cos(\theta) \cdot y_E \sin(\theta)$$
$$x_R = x_E \cos(\theta) \cdot y_E \sin(\theta)$$

4.1.3. **Traslación:** Se suma la coordenada del centro geométrico actual (Cx, Cy) definido por el clic del usuario.

$$x' = x_R + C_x$$
$$y' = y_R + C_y$$

4.2. Proyección Ortogonal de un Punto sobre un Segmento

Una de las funcionalidades interactivas más complejas es proyectar el punto del clic derecho $P(P_{x,}P_y)$ sobre el primer segmento del polígono estrellado, definido por los vértices A y B. El sistema utiliza cálculo vectorial:
Se definen los vectores AB = B – A y AP = P - A. El factor de proyección escalar t se obtiene mediante el producto punto:

$$t = \frac{AP \cdot AB}{|AB|^2}$$

Para garantizar que la proyección caiga estrictamente dentro del segmento (y no en la línea infinita que lo contiene), el algoritmo restringe t al intervalo [0, 1]. El punto proyectado final P' se calcula como:
P' = A + t.AB

5. Algoritmos y Lógica de Implementación

5.1.Generación de Polígonos y Estrellas

El algoritmo para generar polígonos estrellados itera en un ciclo desde i = 0 hasta 2n (donde n es el número de puntas). En cada iteración par calcula un vértice usando el radio exterior, y en cada iteración impar usa el radio interior.
El ángulo base para cada vértice se calcula dividiendo la circunferencia completa en radianes:

$$\theta_i = i \cdot \left(\frac{\pi}{n}\right)$$

Las coordenadas base de cada vértice antes de aplicar las transformaciones descritas en el punto 3.1 se obtienen mediante trigonometría básica:

$$x = r \cdot cos(\theta_i)$$
$$y = r \cdot sin(\theta_i)$$

5.2.Renderizado Gráfico (GDI+)

El evento `Canvas_Paint` limpia el área de dibujo y ejecuta el método `.Dibujar()` polimórfico de la `figuraActual`. Para lograr bordes suaves de alta calidad, el motor activa el

Anti-Aliasing:`e.Graphics.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.AntiAlias;`

6. Pruebas y validación (Testing)

El módulo `ValidadorMatematico` inyecta casos extremos para certificar la estabilidad de la lógica matemática. Se evalúan:

- Ángulos negativos o superiores a 360° para verificar la normalización trigonométrica.
- Cálculo de áreas (ej. la fórmula del área del hexágono

$$A = \frac{3\sqrt{3}}{2} r^2$$

- Proyecciones con el punto P ubicado exactamente en uno de los vértices (debe devolver el mismo vértice) o muy lejos (el parámetro t debe ser fijado en 0 o 1).

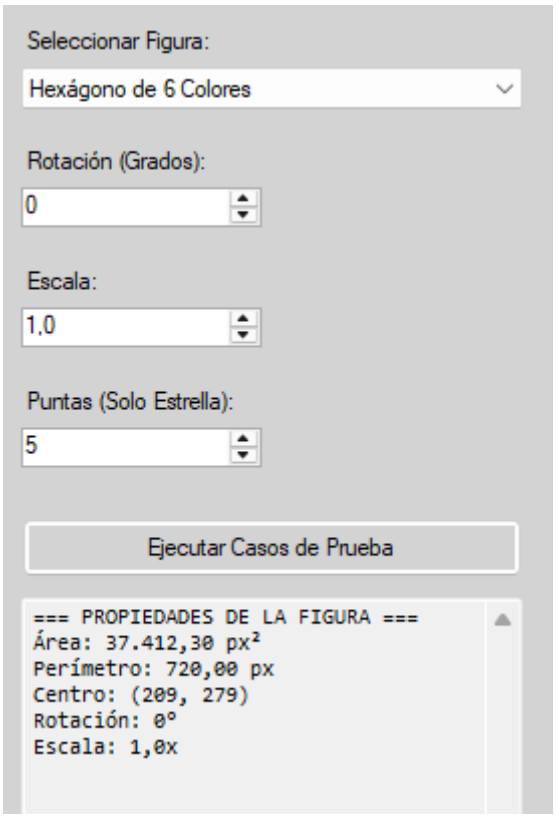


Figura 1 “Salida de resultados”

7. Manual de Usuario y Evidencia Gráfica

7.1. Selección y modificación de figuras

El usuario puede escoger entre Hexágono, Flor y Estrella mediante la lista desplegable. Al usar los controles numéricos (*SpinBoxes*) de Escala y Rotación, la figura reacciona en tiempo real redibujándose en el lienzo sin rastro residual.

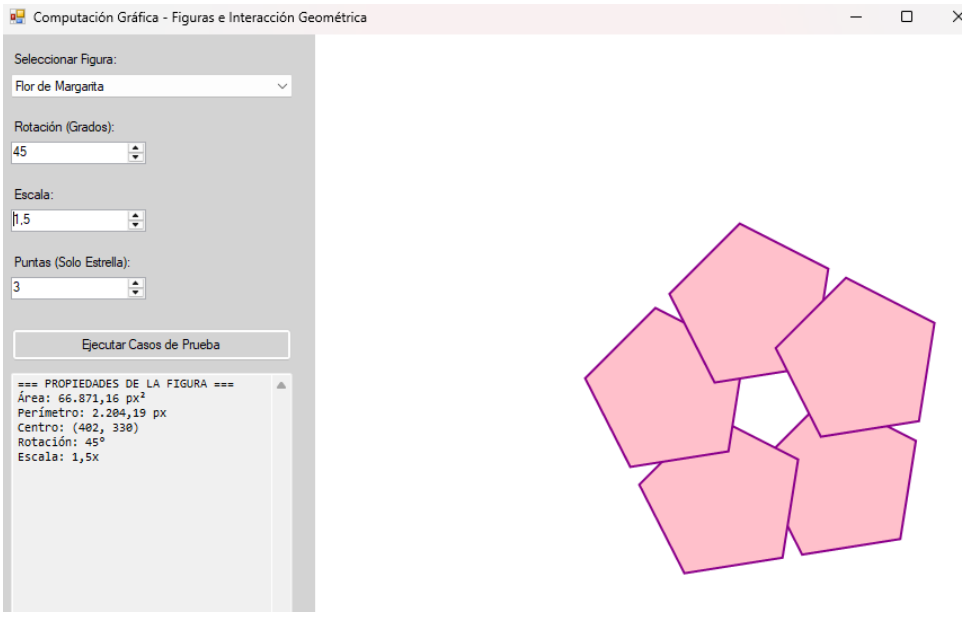


Figura 2. Flor Margarita

7.2.Interacción con el ratón (Traslación)

El usuario puede cambiar el origen de coordenadas cartesianas globales haciendo **clic izquierdo** en cualquier parte del lienzo en blanco. Esto actualiza de inmediato el centro de la figura y su información paramétrica en pantalla.

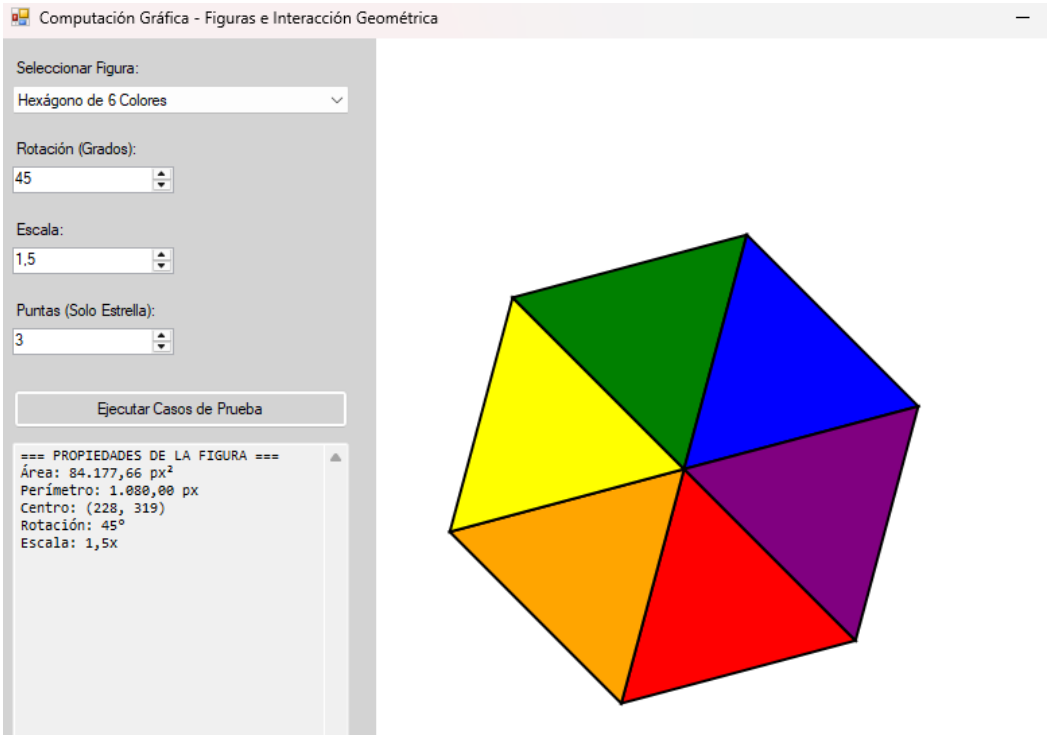


Figura 3. Hexágono de 6 colores

7.3.Interacción geométrica: proyección de vector

Al seleccionar la figura "Polígono Estrellado" y hacer **clic derecho** en cualquier espacio del lienzo, se despliega una línea segmentada que calcula visualmente la distancia más corta (perpendicular) hacia el primer segmento de la figura.

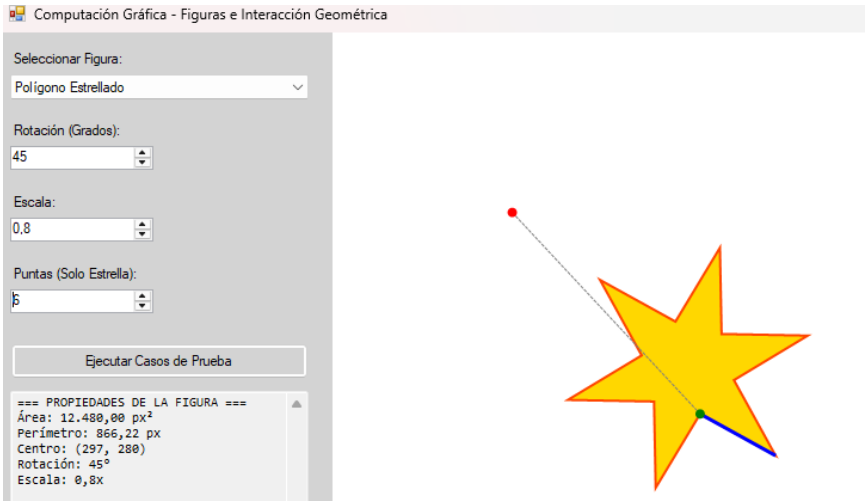


Figura 4. Polígono estrellado

8. Conclusiones

Sobre la Construcción de Figuras: La generación puramente algorítmica basada en funciones trigonométricas (cos, sen) demostró ser completamente efectiva para estructurar el hexágono regular dividido, la flor simétrica de pentágonos y las estrellas de N puntas, permitiendo cambiar dimensiones y geometrías de forma dinámica sin perder la proporción original.

Sobre el Motor Matemático: La creación de la clase `GeometryMath` y el tipo `Vector2D` aislados de la UI permitió calcular eficientemente el área (mediante la fórmula de Shoelace) y la proyección ortogonal de un punto sobre un segmento. La separación de la lógica matemática garantizó la reutilización del código y simplificó la verificación de resultados.

Sobre las Transformaciones Matriciales: La implementación de matrices compuestas en coordenadas homogéneas (3 X 3) permitió aplicar rotaciones y escalamientos respecto a un centro dinámico de manera limpia y uniforme. Se comprobó experimentalmente la propiedad de invarianza: trasladar o rotar una figura conserva intactos su perímetro y su área original.

Sobre la Interfaz y Validación: La arquitectura en capas con GDI+ y el uso de técnicas como el doble búfer aseguraron una navegación fluida a 60 FPS durante la interacción con el ratón. Asimismo, la inclusión del módulo `ValidadorMatematico` certificó la exactitud de los algoritmos mediante pruebas unitarias internas ejecutables directamente desde la interfaz.

Video:
https://drive.google.com/file/d/1P1bazWzv9xCVn2JYRU5br_59r55gkcQ_/view?usp=sharing